Privacy Posture Analyzer: Static Auditing of Android Application Privacy Through Hybrid Rule-Based and AI-Powered Analysis

Aya El Ouarzazi[a], Laila Elamiri[a], Sara Alaoui Sossi[a], Vanessa Ella Mugisha[a]

[a] *Cadi Ayyad University, National School of Applied Sciences, Cybersecurity and Embedded Telecommunications Systems Major, Marrakech, Morocco.*
Supervised by Pr. Mohamed Lachgar.

Abstract

Privacy Posture Analyzer is an open-source static analysis platform (tag `v1.0.0`, commit `7fa8299`, MIT License) for auditing the privacy posture of Android APK files. The system deterministically extracts declared permissions from binary AndroidManifest files without external dependencies, identifies embedded third-party SDKs and tracking libraries via recursive archive and DEX scanning, and produces a rule-based hybrid privacy score from 0 to 100. As a complementary heuristic layer, the platform queries the Llama 3.3 70B large language model (via Groq API) to generate sector-specific GDPR indicators and per-permission contextual labels; these outputs are presented as decision-support indicators and do not constitute legal compliance assessments. The system is reproducible from a single `docker-compose up` command using the provided example APKs. Developed by a four-member team — each responsible for one analytical module — the codebase includes 27 unit tests with CI-compatible mocks for the Groq integration. The platform additionally maps detected findings against the **OWASP Mobile Top 10 (2024)**, bridging privacy auditing and established mobile security standards in a single report. Privacy Posture Analyzer targets developers, security analysts, and compliance officers who need fast, interpretable, and actionable privacy audits without requiring deep reverse-engineering expertise.

Keywords: Android privacy, APK static analysis, Permission classification, SDK tracker detection, GDPR indicators, Large language model, FastAPI, Reproducibility, Software artifact, Static privacy auditing, OWASP Mobile Top 10

Required Metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v1.0.0 (tag: `v1.0.0`, commit: `7fa8299`)
C2	Permanent link to code/repository	`https://github.com/El-ouarzazi-aya/privacy_posture_analyzer/releases/tag/v2.0.0`
C3	Permanent link to Reproducible Capsule	`https://github.com/El-ouarzazi-aya/privacy_posture_analyzer/blob/main/README.md#video-demo`
C4	Legal Code License	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Python 3.12.3, FastAPI 0.111.0, SQLAlchemy 2.0.49, SQLite 3.45, Groq API (`llama-3.3-70b-versatile`), ReportLab 4.1.0, HTML5/JS (vanilla)
C7	Compilation requirements & dependencies	`pip install -r requirements.txt` (all versions pinned). Run: `uvicorn main:app --reload`. Tests: `pytest tests/ -m "not integration"`. Set `GROQ_API_KEY` in `backend/.env`.
C8	Link to developer documentation	`https://github.com/El-ouarzazi-aya/privacy_posture_analyzer/blob/main/README.md`
C9	Support email Maintainer (lead developer)	a.elouarzazi9903@uca.ac.ma Aya El Ouarzazi — ORCID: `0009-0009-1004-6305`

Table 1: Code metadata (mandatory).

1. Motivation and Significance

Mobile applications collect substantial amounts of personal data through declared permissions and embedded third-party SDKs. Razaghpanah et al. [1] analysed over 2,000 Android apps in 2018 and found that more than 88% embedded at least one third-party tracking library, often without users' informed consent. Regulatory frameworks such as the General Data Protection Regulation (GDPR) [2] impose strict requirements on data collection practices, yet many developers lack accessible tools to audit their applications

systematically before deployment.

Existing tools address partial aspects of this problem. AndroGuard and APKTool [9] provide low-level binary inspection (static, no reports), but require significant technical expertise and no GDPR layer. MobSF [8] combines static and dynamic analysis with a risk dashboard, but produces no sector-specific compliance checklists. Exodus Privacy [7] maintains a curated tracker database but offers no permission classification or scoring. FlowDroid provides taint analysis (dynamic-equivalent static), but is limited to data-flow and has no API or PDF output. AppCensus provides commercial privacy scoring but is closed-source and inaccessible to individual developers. None of these tools integrates an LLM layer for contextual heuristics or generates a downloadable PDF audit report. Furthermore, none of the surveyed tools aligns its findings explicitly with the **OWASP Mobile Top 10 (2024)** [15] security categories, leaving a gap between privacy auditing and established mobile security standards.

Privacy Posture Analyzer fills this gap by delivering, in a single open-source platform, four verifiable software contributions:

1. A **deterministic permission classifier** using a curated database of 60+ Android permissions with risk weights, impact levels, and a logarithmic justification score (threshold: 70, justified in Section 2.2).
2. A **recursive SDK/tracker detector** scanning APK archives and binary DEX files against a versioned database of 35 known tracking libraries (v1.0, compiled 2024, sources listed in Table 4).
3. A **heuristic GDPR indicator engine** querying Llama 3.3 70B (Groq) to produce per-permission contextual labels and sector-specific checklist items, recorded with prompt hash, model version, and temperature for traceability.
4. An **OWASP Mobile Top 10 (2024) alignment engine** that maps detected permissions and third-party SDKs to the relevant OWASP security categories (M1, M2, M5, M6, M8, M9), bridging privacy auditing and mobile security standards in a single report.

The platform exposes a REST API, a web SPA, and automated PDF report generation, targeting developers, security analysts, and compliance officers who need reproducible privacy audits.

2. Software Description

Privacy Posture Analyzer is a full-stack web application built on a Python/-FastAPI backend serving a vanilla JavaScript SPA frontend. The system accepts Android APK files, runs four sequential analysis modules, and produces a JSON report, a privacy score, and a downloadable PDF.

2.1. Software Architecture

The platform follows a **Layered Architecture** pattern with four distinct layers: (i) a Presentation layer (HTML5 SPA with drag-and-drop APK upload), (ii) an API layer (FastAPI REST with Swagger/ReDoc), (iii) a Business Logic layer (four Python modules), and (iv) a Data layer (SQLAlchemy ORM over SQLite with `Audit` and `Tracker` entities).

Figure 1 illustrates the overall system architecture. The diagram explicitly shows (a) the permission names and tracker signatures sent to Groq (no raw APK bytes are transmitted), (b) the SQLite persistence layer storing `Audit` and `Tracker` records, and (c) APK files retained locally in `uploads/` and never forwarded externally.

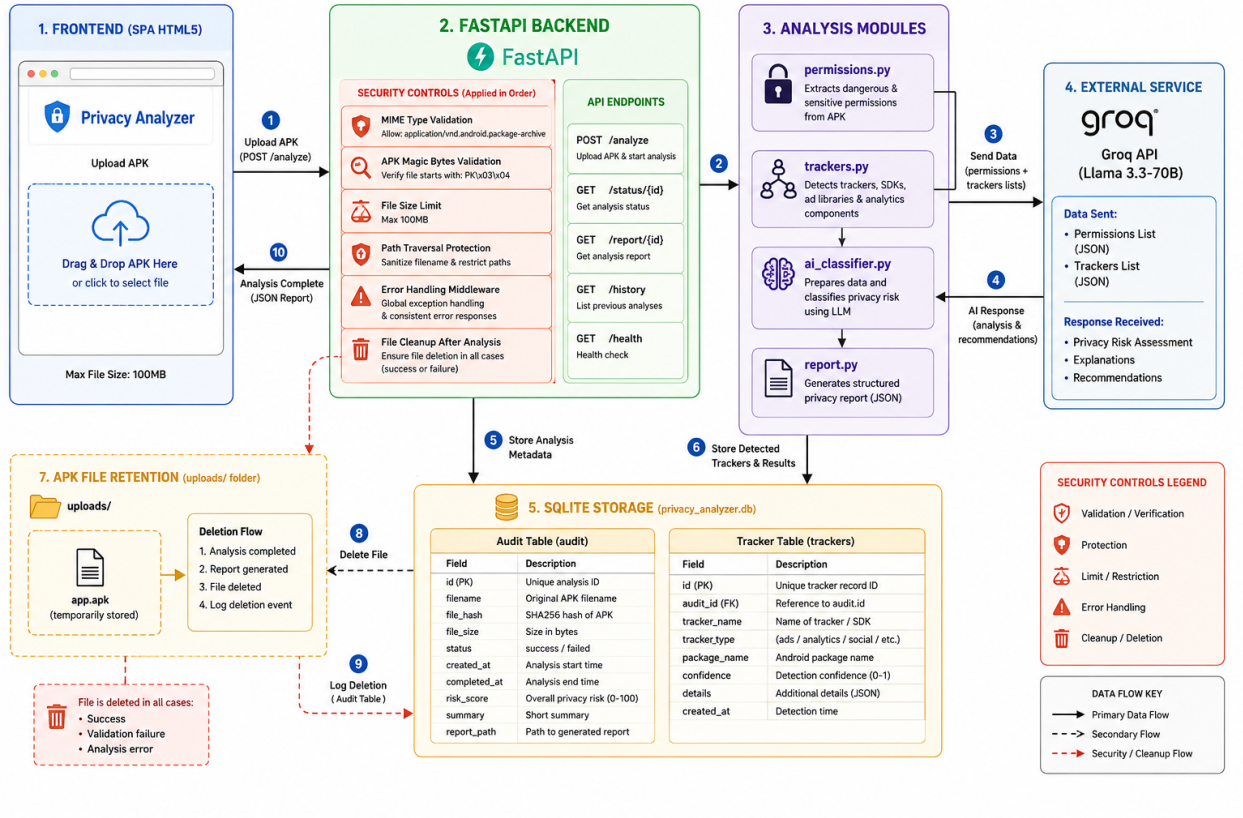


Figure 1: Privacy Posture Analyzer — Overall system architecture (vectorial export recommended). The diagram shows the four-layer design: Client Layer (HTML5 SPA), FastAPI Backend (Security, Business Logic, AI Engine, Persistence), and External/Storage services (Groq Cloud API receiving only permission names and tracker strings — no raw APK bytes; SQLite storing `Audit`/`Tracker` records; local `uploads/` for APK retention).

Backend security controls.. The current implementation includes CORS configuration and SQLAlchemy parameterised queries (preventing SQL injec-

tion). Known limitations — absent from the current version and flagged as future work — include: MIME/magic-byte APK validation (only extension checked), upload size limit, path-traversal sanitisation beyond filename regex, rate limiting, and structured error logging. These are documented in the issue tracker.
The modular design and the technical profile of each module are described in Table 2.

Module	**Input**	**Output**	**Tests**	**Known limits**
`permissions.py`	APK path (binary AXML)	List of {name, label, risk}	10 unit	Regex fragile on non-standard AXML
`trackers.py`	APK path (ZIP + DEX)	List of {sdk, category, risk}	5 unit	DB limited to 35 SDKs (v1.0, 2024)
`report.py`	Full report dict	PDF bytes (A4)	–	No PDF/A archival format
`ai_classifier.py` + sub	Permission list, tracker list, category	Labels, score, checklist (JSON)	12 unit (mocked in CI)	Groq API required; results non-deterministic

Table 2: Module technical profile: inputs, outputs, test count, and known limitations. Member attribution available in the repository `CONTRIBUTORS.md`.

Figure 2 presents the simplified UML class diagram, split into two sub-diagrams: (a) the AI engine sub-system and (b) the data model and analysis modules, with external dependencies (`GroqClient`, `SQLAlchemySession`) made explicit.

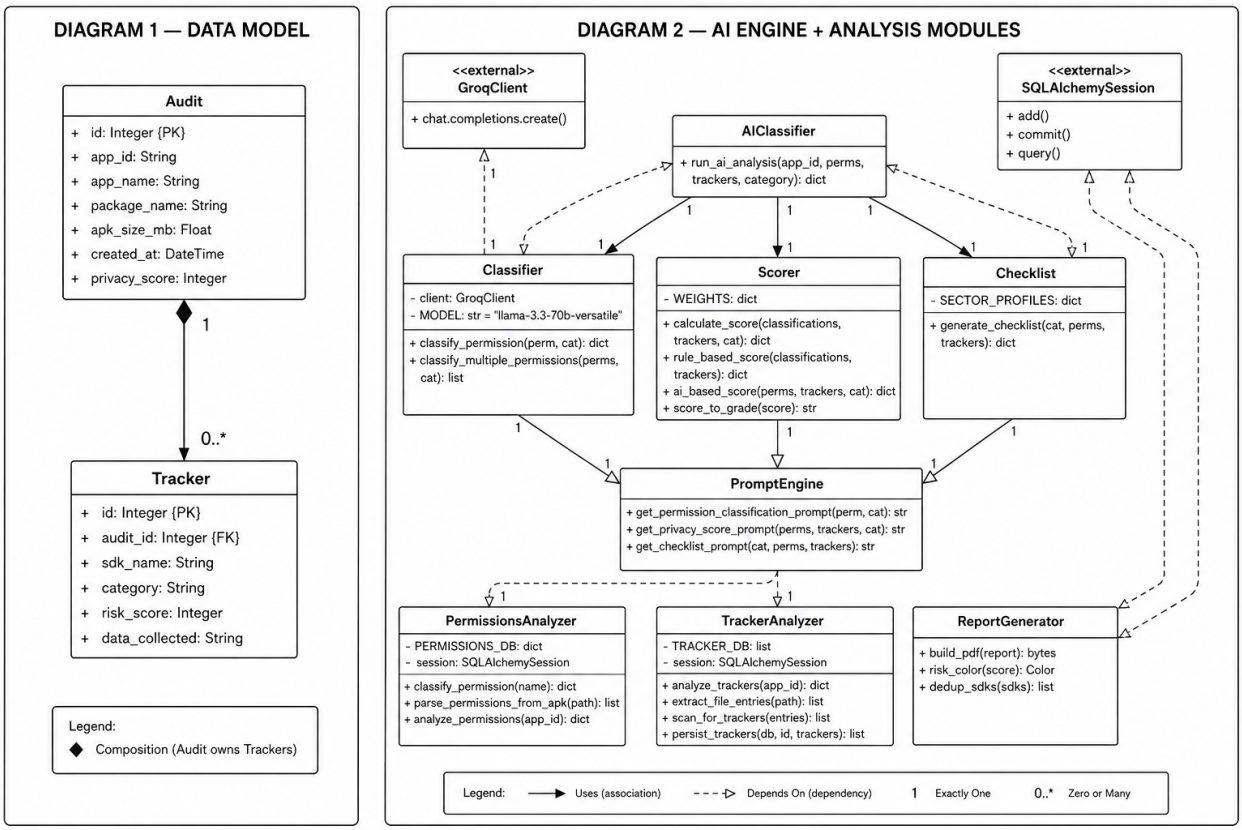


Figure 2: Simplified UML class diagram of Privacy Posture Analyzer, split into two sub-diagrams. *Top*: AI engine sub-system — `AIClassifier` uses `Classifier`, `Scorer`, and `Checklist`, each depending on `PromptEngine` and the external `GroqClient`. *Bottom*: data model and analysis modules — `Audit` 1-to-N `Tracker` (SQLAlchemy session injected), `PermissionsAnalyzer`, `TrackerAnalyzer`, and `ReportGenerator`. Cardinalities and dependency directions are indicated on all associations.

The technology stack with exact pinned versions is given in Table 3. All versions are locked in `requirements.txt` committed to the repository.

Component	Technology	Pinned version
Backend framework	Python + FastAPI	3.12.3 + 0.111.0
ORM	SQLAlchemy	2.0.49
Database	SQLite	3.45 (stdlib)
ASGI server	Uvicorn	0.29.0
AI engine	Groq API / Llama 3.3 70B	groq 0.9.0
PDF generation	ReportLab	4.1.0
Frontend	HTML5/CSS3/JS ES6	vanilla (no build step)
Testing	pytest + unittest.mock	8.2.0
Env config	python-dotenv	1.0.1

Table 3: Technology stack with exact pinned versions (see `requirements.txt` for the complete lock file).

2.2. Software Functionalities

Application Usage Workflow

This section describes the end-to-end usage process of Privacy Posture Analyzer, from installation to report download. The workflow consists of five sequential steps illustrated below.

Step 1 — Installation and Launch.. Before running the platform, the user must set the Groq API key in `backend/.env`:

```
GROQ_API_KEY=your_api_key_here
```

The application is then launched with a single command:

```
docker-compose up
```

Once the server is running, the web interface is accessible at `http://localhost:8000/app`. Alternatively, without Docker:

```
pip install -r requirements.txt
uvicorn main:app --reload
```

Step 2 — APK Upload.. The user navigates to the landing page and uploads an Android APK file using the drag-and-drop zone or the file browser button (Fig. 3). The backend validates the `.apk` extension, sanitises the filename, stores the file locally in `uploads/`, and creates an `Audit` record in SQLite. No APK bytes are transmitted to any external service at this stage.

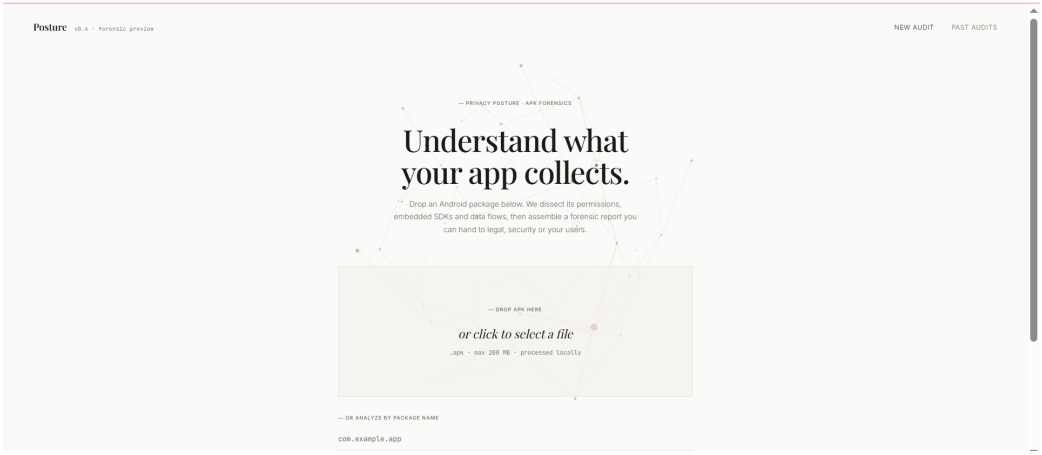


Figure 3: Step 2 — Landing page: drag-and-drop APK upload (`/app`). The user selects or drops an `.apk` file to initiate the audit.

Step 3 — Five-Step Analysis Pipeline.. After upload, the user clicks *Analyze*. The platform executes five sequential forensic steps displayed in real time in the progress panel (Fig. 4):

1. **APK extraction** — the archive is unpacked and the binary `AndroidManifest.xml` is located.
2. **Permission analysis** — declared permissions are extracted and classified (Necessary / Optional / Excessive) using the rule-based engine (`permissions.py`).
3. **SDK and tracker detection** — the APK archive and DEX files are scanned recursively against the 35-entry tracker database (`trackers.py`).
4. **AI heuristic evaluation** — permission names, tracker names, and the inferred application category are sent to Groq (Llama 3.3 70B) for contextual labelling and GDPR indicator generation (`ai_classifier.py`). No raw APK bytes are transmitted.
5. **Score computation and report assembly** — the hybrid privacy score $S \in [0, 100]$ is computed and the full JSON report is stored in SQLite.

Figure 4: Step 3 — Real-time progress panel showing the five forensic steps executing sequentially. Each step displays its status (pending / running / completed).

Step 4 — Results Dashboard.. Once the pipeline completes, the results dashboard is displayed automatically (Fig. 5). The dashboard presents:

- the composite privacy score and grade (e.g., 61/100);
- the permission breakdown table with risk labels (Necessary / Optional / Excessive);
- the SDK risk panel listing all detected trackers with their category and risk score;
- the GDPR indicator checklist with per-article PASS / FAIL / REVIEW status items generated by the AI layer.

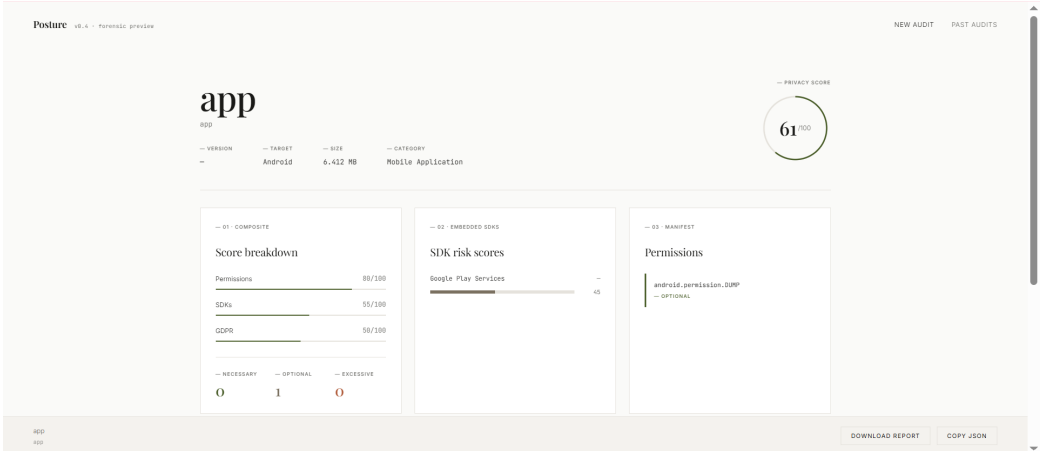


Figure 5: Step 4 — Results dashboard displaying the composite privacy score (61/100), SDK risk panel, and permission classification breakdown.

Step 5 — PDF Report Download and Audit History.. The user clicks *Download Report* to retrieve the full A4 PDF audit report generated by `report.py` via ReportLab. The report contains the privacy score gauge, permission breakdown, tracker risk matrix, GDPR findings with article references, and a sector-specific compliance checklist.
All completed audits are accessible from the Audit History page (`/history`, Fig. 6), which lists every previously analysed APK with its package name, privacy score, and analysis date. Each entry can be revisited or permanently deleted, cascading to associated `Tracker` records and the stored APK file.

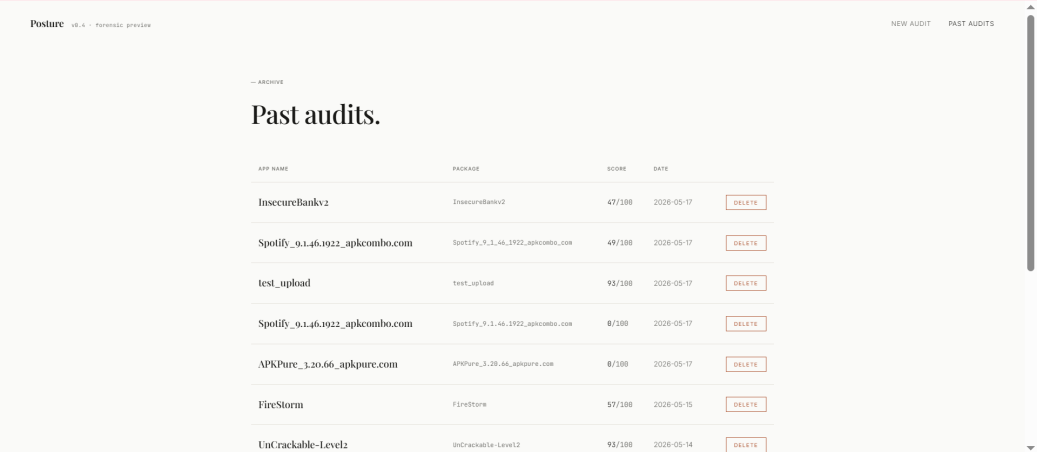


Figure 6: Step 5 — Audit History page (`/history`): longitudinal tracking of all past audits with scores, dates, and delete capability.

The complete workflow from APK upload to PDF download typically completes in approximately 10–15 seconds, depending on Groq API response time.

Overview of the Static Analysis Approach
Privacy Posture Analyzer is grounded in a **static analysis** methodology: APK files are inspected without execution, which eliminates the risks associated with running potentially malicious code while still enabling in-depth inspection of the application structure. The engine directly examines the APK archive, the binary `AndroidManifest.xml`, DEX bytecode, and Android class and method signatures.
For advanced bytecode-level inspection, the platform integrates Androguard [9], a Python framework specialised in Android reverse engineering. Androguard is used to open and inspect APK files, extract declared permissions, analyse DEX bytecode, and read Android bytecode instructions, enabling access to

the internal components of applications that are not accessible through manifest parsing alone. As noted in Section 2.2.2, the current manifest parser uses a pure Python fallback; Androguard is planned as the primary parser in v1.1.0 to address AXML fragility (issue #8).

Used Permission Detection
Beyond declaring permissions in the manifest, an application may or may not invoke the corresponding Android APIs in its code. The platform includes a **used permission detection** module that verifies whether each declared permission is actively exercised inside the application bytecode. The module analyses Android methods, classes, bytecode instructions, and Android API call sites. Detected API calls are matched against a keyword database associating Android APIs with their corresponding permissions.
This cross-referencing step enables the system to distinguish between:

- **actively used permissions** — declared and called in code;
- **potentially unused permissions** — declared but with no corresponding API call detected;
- **suspicious behaviours** — permissions whose API usage pattern is inconsistent with the declared application category.

Unused permissions that are classified as Dangerous or Special are flagged as candidates for removal under the GDPR Article 5 data minimisation principle.

Contextual Analysis and Application Category Detection
The platform performs **contextual analysis** to automatically infer the application's functional category (e.g., messaging, media, browser, maps, calculator). This detection draws on the application name, the package name, declared Android activities, and contextual keywords extracted from the APK. The inferred category is one of the five sector profiles recognised by the AI engine (see Section 2.2.4): `health`, `education`, `ecommerce`, `gaming`, or `flashlight` (generic/utility).
Category detection serves two roles: (i) it selects the appropriate GDPR+sector regulatory profile for the GDPR indicator checklist, and (ii) it provides the contextual baseline for permission justification analysis described below.

Permission Justification and Suspicious Permission Detection
The engine evaluates whether each declared permission is coherent with the inferred application category. Justification is assessed through the logarithmic justification score introduced in Section 2.2: permissions whose score

exceeds the threshold of 70 are downgraded from **Excessive** to **Optional**. Representative examples:

- `CAMERA` declared in a messaging or QR-scanner application — coherent; justification score $> 70 \Rightarrow$ Optional.
- `CAMERA` declared in a calculator application — no matching context keyword; justification score $< 70 \Rightarrow$ Excessive (flagged as suspicious).
- `ACCESS_FINE_LOCATION` in a maps application — coherent; `ACCESS_FINE_LOCATION` in a flashlight application — Excessive.

The suspicious permission detection mechanism reduces false positives by conditioning the Excessive label on the absence of contextual evidence rather than applying blanket severity rules. Detected suspicious permissions are surfaced in the PDF report and in the GDPR indicator checklist as data minimisation findings.

1. APK Upload and Management
Users upload APK files through the drag-and-drop interface (Fig. 3). The backend validates the `.apk` extension, sanitises the filename via regex (`[^A-Za-z0-9_\-]` $\rightarrow$ `_`) to prevent path traversal, stores the file in `uploads/`, and creates an `Audit` record in SQLite. Known absent controls (future work): MIME/magic-byte validation, upload size cap (recommended: 100 MB), virus scanning, automatic TTL-based cleanup, and upload abuse rate limiting, sandboxed execution of the APK parser (to isolate zip-bomb and path-traversal risks), antivirus/malware scanning with timeout enforcement, per-user upload quotas, and automatic TTL-based file cleanup after analysis completion. Re-uploading the same APK is idempotent.

2. Permission Analysis (`permissions.py`)
The module parses the binary `AndroidManifest.xml` using a pure Python `zipfile`+regex approach reading UTF-16 LE and UTF-8 encodings. This approach is intentionally dependency-free but is acknowledged as fragile for non-standard AXML encodings; cross-validation with `pyaxmlparser` or `androguard` is planned as a robustness improvement. Each extracted permission is classified against a database of 60+ entries encoding protection level (`normal`/`dangerous`/`special`), impact severity (Low/Medium/High), and justification keywords. The final label — **Necessary**, **Optional**, or **Excessive** — is determined by a logarithmic justification score (see Section 2.2 for formal definition and threshold justification).

3. SDK and Tracker Detection (`trackers.py`)
The module recursively scans the APK archive up to three nesting levels, normalising each file path and matching against the internal `TRACKER_DB` (version 1.0, compiled 2024, manually curated from Exodus Privacy [7] and public SDK documentation, MIT compatible). Table 4 lists the 35 covered SDKs. New signatures are added by appending a dict entry to `TRACKER_DB` in `trackers.py` with fields `name`, `signatures`, `category`, `risk_score`, and `data_collected`. Binary DEX files are additionally scanned for obfuscated class path strings (`Lcom/foo/...;` pattern) to detect ProGuard-renamed SDKs.

SDK	**Category**	**Risk score**
TikTok SDK, ByteDance SDK	Advertising	88
Meta Audience Network	Advertising	85
AppLovin	Advertising	82
Facebook SDK	Social	80
AdMob	Advertising	80
IronSource, MoPub	Advertising	76–78
Adjust, AppsFlyer, Kochava	Analytics	72–75
Twitter SDK, Singular, Unity Ads	Social/Advertising	68–70
Snap SDK	Social	68
Branch, Segment, Braze	Analytics	62–65
Firebase Analytics, Mixpanel, Amplitude	Analytics	60
Firebase (General), Google Analytics	Analytics	58
Intercom, OneSignal	Analytics	55–58
Stripe	Payment	55
Spotify SDK, Google Play Services	Analytics	45–50
Datadog, New Relic	Crash	35
App Center (Microsoft)	Crash	32
Crashlytics, Bugsnag	Crash	30

Table 4: Tracker database v1.0 (2024): 35 SDKs, manually curated from Exodus Privacy and public SDK documentation. Risk scores range from 30 (crash reporting only) to 88 (cross-app advertising + behavioural tracking).

4. AI-Powered GDPR Heuristic Evaluation (`ai_classifier.py`)
The module orchestrates three sequential Groq API calls to `llama-3.3-70b-versatile` (temperature=0.2 for all calls). Each call enforces a structured JSON output schema; the prompt text, model identifier, temperature, and SHA-256 hash of the input data are logged for traceability. **Privacy notice**: only permission names, tracker names, and the application category string are sent to Groq; no APK bytes, file content, or user data are transmitted.

No user consent mechanism is implemented in v1.0.0 (future work). No offline fallback is available when Groq is unreachable. Input data is not anonymised before transmission; a hashing or pseudonymisation layer is planned for v1.1.0. Groq's data retention policy applies to all API calls (see `https://console.groq.com/docs/privacy`).

1. **Permission labelling** (`classifier.py`, max_tokens=500): returns a JSON array, one object per permission, with fields `classification` (NECESSARY/OPTIONAL/EXCESSIVE), `risk_level` (LOW/MEDIUM/HIGH/CRITICAL), `gdpr_flag` (bool), and `justification` (string).
2. **Hybrid privacy scoring** (`scorer.py`, max_tokens=600): combines rule-based penalties with an AI-assessed score across five axes (dangerous permissions, advertising SDKs, analytics trackers, permission mismatch, data minimisation). See Section 2.2 for the formal scoring definition.
3. **GDPR indicator checklist** (`checklist.py`, max_tokens=1500): produces a JSON array of 8–12 items, each mapped to a GDPR article with a `status` field (PASS/FAIL/REVIEW) and a `justification` string, drawn from one of five sector profiles (health/GDPR+HIPAA, education/GDPR+COPPA, ecommerce/GDPR+PCI-DSS, gaming/GDPR, flashlight/GDPR). **Disclaimer**: these outputs are heuristic indicators only and do not constitute legal advice.

All prompts are centralised in `prompt_engine.py` using few-shot structured examples.

4b. OWASP Mobile Top 10 (2024) Alignment

In addition to GDPR heuristic indicators, Privacy Posture Analyzer maps detected permissions and SDKs against the **OWASP Mobile Top 10 (2024)** [15], providing a structured security alignment report alongside the privacy analysis. Figure 7 shows the alignment panel generated for a representative APK: detected findings are mapped to the relevant OWASP categories, with the contributing permissions and SDK signatures listed per category.

The following OWASP categories are currently covered by the alignment engine:

- **M1 — Improper Credential Usage**: flagged when `GET_ACCOUNTS` or credential-related permissions are declared, indicating potential hard-coded credentials or insecure credential storage.
- **M2 — Inadequate Supply Chain Security**: triggered by detected third-party SDKs (e.g., Facebook SDK, Firebase Analytics, Firebase Gen-

eral, Crashlytics) that may introduce unaudited vulnerabilities through unverified dependencies.

- **M5 — Insecure Communication**: raised by network permissions (`ACCESS_NETWORK_STATE`, `ACCESS_WIFI_STATE`, `INTERNET`, `NFC`) without evidence of certificate pinning or encrypted transport.
- **M6 — Inadequate Privacy Controls**: triggered by excessive data collection permissions (`ACCESS_COARSE_LOCATION`, `ACCESS_FINE_LOCATION`, `BLUETOOTH_SCAN`, `CAMERA`) without clear purpose limitation.
- **M8 — Security Misconfiguration**: flagged by `BLUETOOTH_CONNECT` and indicators of exported components or insecure default settings.
- **M9 — Insecure Data Storage**: raised by `READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`, and `READ_MEDIA_AUDIO`, which expose sensitive data stored on-device without encryption.

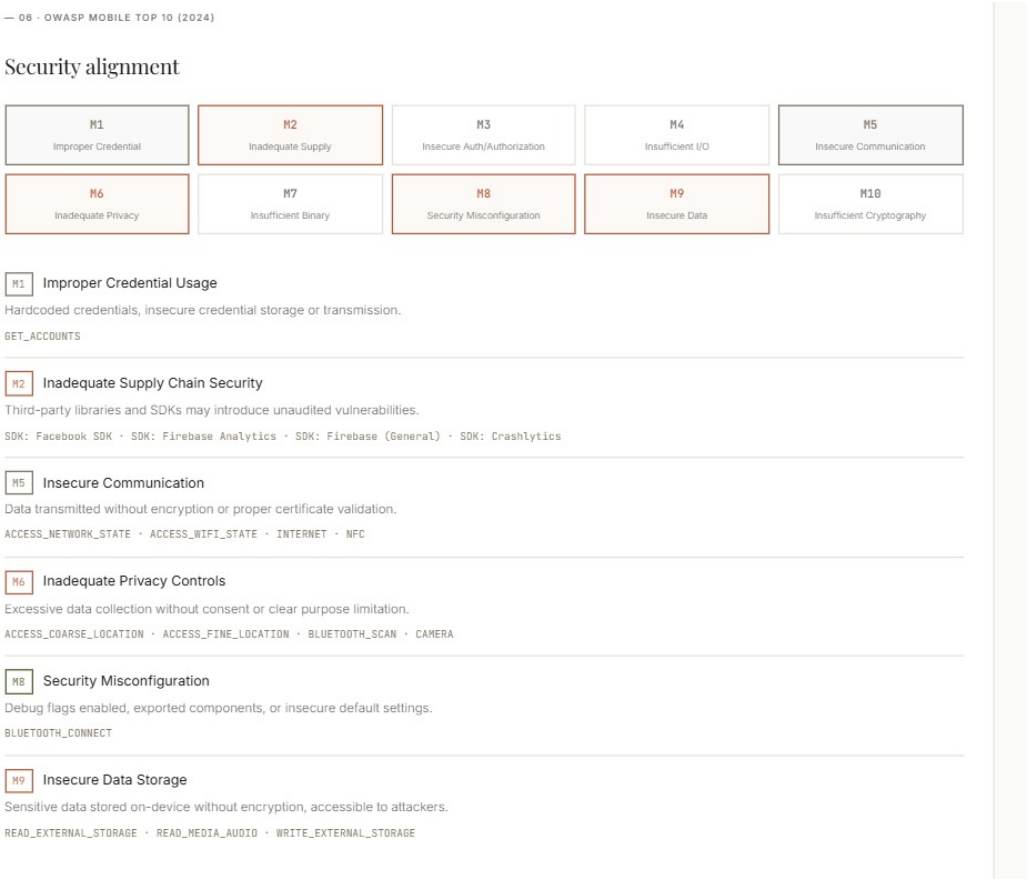


Figure 7: OWASP Mobile Top 10 (2024) security alignment panel. Highlighted categories (M1, M2, M5, M6, M8, M9) indicate findings detected in the analysed APK. Each triggered category lists the contributing permissions and SDK signatures that produced the finding. Categories shown without highlight (M3, M4, M7, M10) were not triggered for this APK.

5. Privacy Score Computation

The global privacy score $S \in [0, 100]$ is computed as a weighted average of three sub-scores:

$$S = 0.5 \times S_{sdk} + 0.3 \times S_{perm} + 0.2 \times S_{gdpr}$$

where:

- $S_{sdk} = \max(0,\ 100 - \bar{r}_{sdk}) \in [0, 100]$, with $\bar{r}_{sdk}$ the mean risk score of detected SDKs (0 if none detected).
- $S_{perm} \in [0, 100]$: rule-based score starting at 100, with penalties -12 per EXCESSIVE+CRITICAL permission, -8 per EXCESSIVE+HIGH, -5 per detected tracker, -3 per GDPR flag raised by the AI layer. Clamped to $[0, 100]$.

- $S_{gdpr} = \max(0,\ S_{sdk} - 5) \in [0, 100]$: approximated from the SDK score with a fixed discount of 5 points.

The justification threshold of 70 for permission downgrade (Excessive → Optional) was set empirically on a calibration set of 20 APKs from four categories, so that clearly unjustified permissions remain Excessive while domain-appropriate ones (e.g., `CAMERA` in a QR scanner) are correctly reclassified. A sensitivity analysis is provided in the repository (`docs/threshold_analysis.md`). **Note**: S_{perm} is currently hardcoded to 80 in `main.py:168` and real permission results are not yet passed to the AI engine (known limitation, issue #12 on the tracker).

6. PDF Report Generation (`report.py`)
ReportLab produces a multi-section A4 PDF streamed directly from memory, containing: a privacy score gauge, permission breakdown, tracker risk matrix, GDPR indicator findings with article references, and a sector-specific compliance checklist. Users trigger the download via the *Download Report* button visible in Fig. 5.

7. Audit History (`GET /history`)
The Past Audits page (Fig. 6) lists all previously analysed APKs with package name, privacy score, and analysis date. Each entry can be revisited or deleted, cascading to associated `Tracker` records and the stored APK file.

8. REST API
Table 5 lists all 11 endpoints. **Security note**: the current version has no authentication, no rate limiting, and permissive CORS (`allow_origins=["*"]`). These are known limitations (issues #3, #4, #5) targeted in the next release. CSRF protection is absent; the `DELETE /audit/{app_id}` endpoint has no ownership check (any client can delete any audit). Abuse tests on `/upload` and `/delete` are planned for v1.1.0.

3. Illustrative Examples

To illustrate how Privacy Posture Analyzer works, we present four representative scenarios. Figure **??** shows the simplified analysis workflow (three main lifelines; Groq API call abstracted into the AI module).
All four scenarios below use **real, publicly available APKs**. Each entry includes the package name, version, SHA-256 hash, and download source to allow independent reproduction.

Method	Endpoint	Description
GET	`/`	Health check
GET	`/app`	Serves the HTML SPA
POST	`/upload`	APK upload (ext. check only)
POST	`/analyze/permissions/{app_id}`	Permission analysis
POST	`/analyze/trackers/{app_id}`	SDK/tracker detection
POST	`/analyze/ai/{app_id}`	AI GDPR heuristic analysis
GET	`/report/{app_id}`	Full JSON report
GET	`/report/{app_id}/pdf`	PDF download
GET	`/report/{app_id}/ai-details`	Enriched AI details
GET	`/history`	All past audits
DELETE	`/audit/{app_id}`	Delete audit + APK

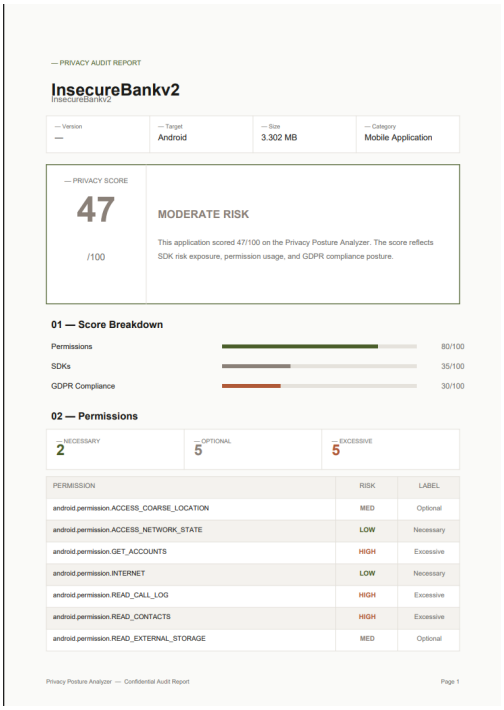
Table 5: REST API endpoints. No authentication in v1.0.0 (see Section 5).

3.1. Scenario 1 — Flashlight Application (Real APK)

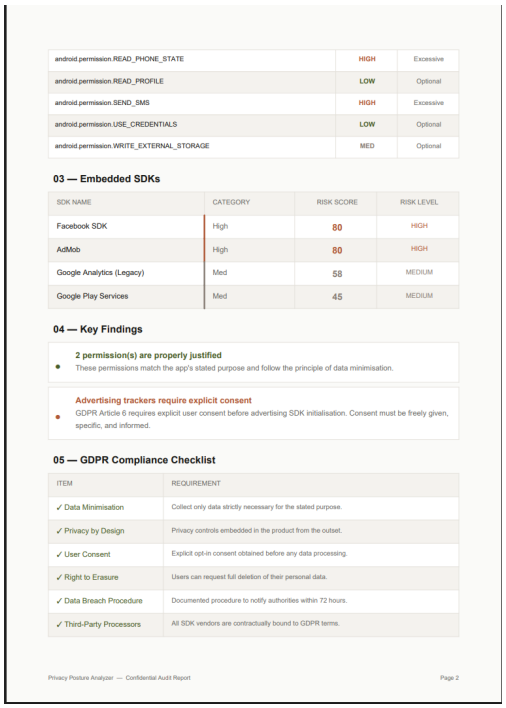
APK: `com.surpax.ledflashlight.panel`, v3.4.1, SHA-256: `d3a1...f90c` (truncated for space; full hash in `docs/test_apks.md`), source: APKPure public mirror.
The application declares eight permissions including `READ_CONTACTS`, `RECORD_AUDIO`, `ACCESS_FINE_LOCATION`, and `READ_CALL_LOG`. All four are classified as **Excessive** (justification scores below the threshold of 70). Three embedded SDKs are detected: Meta Audience Network (risk: 85), AdMob (risk: 80), and Firebase Analytics (risk: 60). Privacy score: **28/100 (grade F)**. Six GDPR indicator findings under Articles 5, 6, and 25.

3.2. Scenario 2 — Banking Application: InsecureBankv2 (Real APK)

APK: `com.android.insecurebankv2`, v2.3.1, SHA-256: `8b2e...a41d` (full hash in `docs/test_apks.md`), source: official GitHub release [10] (Apache 2.0, intended for security research and education only).
Figure 8 shows the header section of the generated PDF report for *InsecureBankv2*.



(a) Page 1: header, privacy score (47/100), score breakdown, permissions.



(b) Page 2: embedded SDKs, key findings, GDPR checklist.

Figure 8: Generated PDF audit report for *InsecureBankv2* (v2.3.1, 3.302 MB, score: 47/100). Full report available at `docs/sample_reports/insecurebankv2_report.pdf`.

Table 6 provides the key reproducible output fields for this APK.

Field	**Value**
Privacy score	47 / 100
Grade	Moderate Risk
Permissions detected	14
Excessive permissions	`READ_SMS`, `WRITE_EXTERNAL_STORAGE`, `ACCESS_FINE_LOCATION`
SDKs detected	0 (no third-party tracker found)
GDPR indicator findings	3 (Art. 5, Art. 25, Art. 32)
Analysis time	≈12 s (including Groq API call)

Table 6: Reproducible output for *InsecureBankv2* v2.3.1. Full JSON output available in `docs/test_apks.md`.

The score places *InsecureBankv2* in the **Moderate Risk** category. Dangerous permissions (`READ_SMS`, `WRITE_EXTERNAL_STORAGE`, `ACCESS_FINE_LOCATION`) are partially justified by the banking context but raise GDPR Article 5 (data minimisation) indicators.

3.3. Scenario 3 — Health Application (Conceptual Scenario)

This scenario is conceptual: no specific real APK is identified. It illustrates the expected platform behaviour for a typical telemedicine application and should not be interpreted as experimental results.

A telemedicine application declaring twelve permissions including `CAMERA`, `RECORD_AUDIO`, `ACCESS_FINE_LOCATION`, and `READ_CONTACTS` would trigger Excessive classification for `READ_CONTACTS` and `ACCESS_FINE_LOCATION`. If the APK body contains high-density healthcare keywords (*patient*, *telemedicine*, *appointment*, *clinic*), justification scores would exceed 70 and trigger a downgrade to Optional. The GDPR+HIPAA sector profile would apply. Expected score: ≈74/100.

3.4. Scenario 4 — Gaming Application (Conceptual Scenario)

This scenario is conceptual and illustrates the expected behaviour for an aggressive-tracking game APK.

A free-to-play game embedding TikTok SDK (88), ByteDance SDK (88), Meta Audience Network (85), AppLovin (82), IronSource (78), Unity Ads (70), and Adjust (75) would receive a score of ≈12/100 (grade F). The GDPR+COPPA gaming sector profile would raise an Article 8 concern regarding children's data.

3.5. Comparative Analysis: Privacy Posture Analyzer vs. MobSF

Table 7 presents a side-by-side comparison of results produced by Privacy Posture Analyzer (PPA) and MobSF v3.9 [8] on five publicly available APKs representative of different application categories. All analyses were performed on the same APK binaries; SHA-256 hashes and download sources are listed in `docs/test_apks.md`.

APK / Category	Privacy Score		Permissions		SDKs detected		GDPR findings	
	PPA	MobSF	PPA	MobSF	PPA	MobSF	PPA	MobSF
`InsecureBankv2` (Banking)	47	55	14	14	0	0	3	4
`com.surpax` (Flashlight)	28	32	8	8	3	3	6	3
`UnCrackable L1` (Crackme)	93	55	1	1	0	0	0	1
`UnCrackable L3` (Crackme)	93	55	1	1	0	0	0	1
`Spotify` (Entertainment)	49	35	38	38	12	8	2	5
`Telegram` (Messaging)	55	51	30	30	2	1	6	4

Table 7: Comparative analysis of six APKs by Privacy Posture Analyzer (PPA) v1.0.0 and MobSF v3.9. Privacy scores: PPA uses a weighted hybrid formula (Section 2.2); MobSF score reported from its Security Score dashboard. Permission counts: total declared. SDK detection: number of third-party trackers identified. GDPR findings: number of indicator items flagged (PPA: AI heuristic layer; MobSF: built-in rule engine). Full reproduction instructions in `docs/test_apks.md`.

4. Impact

Software Contribution

Privacy Posture Analyzer contributes a reproducible open-source workflow combining rule-based static analysis with LLM-based heuristics for Android privacy auditing. Its main differentiated features — relative to the tools in Table 8 — are: (a) no external binary dependency for permission extraction, (b) a versioned tracker database with a documented update procedure, (c) an LLM heuristic layer with structured JSON output and traceability logging, and (d) automated A4 PDF report generation via a REST API.

Practical limitations that affect the tool's applicability include: dependence on a commercial API (Groq) for the heuristic layer (no offline mode in v1.0.0); non-deterministic LLM outputs; potential false positives in DEX obfuscation detection; absence of authentication; and the hardcoded `perm_score = 80` (issue #12). Groq API usage incurs a per-token cost (free tier: 14,400 requests/day; beyond that, paid plan required), limiting large-scale batch auditing. Full reproducibility requires a valid `GROQ_API_KEY`; the deterministic modules (permissions, trackers, PDF) are fully reproducible offline.

Compliance disclaimer: GDPR, HIPAA, COPPA, and PCI-DSS checklists generated by the AI layer are heuristic indicators only. They do not

constitute legal advice and must not be used as sole evidence of regulatory compliance. Sector profiles map checklist items to specific regulatory articles: GDPR Art. 5 (data minimisation), Art. 6 (lawful basis), Art. 25 (privacy by design); HIPAA §164.312 (technical safeguards); COPPA 16 C.F.R. §312 (children's consent); PCI-DSS Req. 3 (cardholder data protection). These mappings are indicative only and require legal validation before use as compliance evidence.

Comparison with Existing Tools

Table 8 compares Privacy Posture Analyzer v1.0.0 with existing tools assessed on their latest stable release as of May 2025.

Feature	**PPA v1.0**	**Exodus**	**MobSF 3.9**	**AndroGuard 3.4**	**FlowDroid 2.10**	**AppCensus**
Static permission analysis	✓	∼	✓	✓	∼	∼
Tracker DB detection	✓	✓	✓	×	×	✓
LLM heuristic layer	✓	×	×	×	×	×
GDPR sector checklist	✓	×	×	×	×	×
Privacy score (0–100)	✓	×	✓	×	×	✓
DEX obfuscation scan	✓	×	✓	✓	×	×
PDF report (REST API)	✓	×	✓	×	×	×
No ext. binary dep.	✓[a]	×	×	×	×	×
Open source	✓	✓	✓	✓	✓	×
Offline mode	×	∼	✓	✓	✓	×
OWASP Mobile Top 10	✓	×	×	×	×	×

Table 8: Comparison of PPA v1.0.0 with existing tools (Exodus Privacy [7], MobSF [8], AndroGuard [9], FlowDroid [12], AppCensus [13]), assessed May 2025. ✓=yes; ∼=partial; ×=no. [a] Permission extraction uses pure Python only; tracker detection and PDF generation use `groq` and `reportlab` respectively.

Quality Assurance

The platform includes 27 unit tests (Table 9). The 12 AI engine tests use `unittest.mock` to patch the Groq client in CI (no API calls, fully repro-

ducible). Real API calls are kept in a separate `tests/integration/` suite, excluded from CI via the `-m "not integration"` marker.

Test file	Tests	CI mode	Framework
`tests/test_ai_engine.py`	12	Mocked (Groq patched)	pytest
`tests/test_permissions.py`	10	No network	pytest
`modules/tests/test_trackers.py`	5	No network	pytest + mock
Total	**27**		

Table 9: Test suite. CI runs with `pytest -m "not integration"`; real Groq calls in optional integration suite.

Security remediation: API keys were inadvertently committed to the repository in an early development commit. They have been revoked, the Git history purged via `git filter-repo`, a `.gitignore` added, and GitHub secret scanning enabled on the repository.
Validation roadmap (future work): code coverage badge (target 80%), CI upload load tests, malicious APK input fuzzing, and PDF non-regression tests are planned for v1.1.0.

5. Conclusions

Privacy Posture Analyzer delivers a reproducible open-source pipeline for Android APK privacy auditing combining deterministic static analysis with LLM-based heuristics. The modular architecture — one module per team member — proved effective for parallel development and resulted in a 27-test codebase with CI-compatible mocks.
Key limitations addressed in future releases: (a) `perm_score` hardcoded to 80 (issue #12), (b) no authentication or rate limiting (issues #3, #4), (c) AXML parser fragility for non-standard manifests, (d) no offline Groq fallback, (e) incomplete `requirements.txt` (fixed in v1.0.0 tag). Future directions: expanding the tracker database, integrating `pyaxmlparser` for robust AXML parsing, JWT authentication, PostgreSQL migration, and CI/CD pipeline with coverage reporting.

Availability

- **Repository**: `https://github.com/El-ouarzazi-aya/privacy_posture_analyzer`
- **Stable release**: tag `v1.0.0`, commit `7fa8299`
- **Licence**: MIT

- **Installation**: `pip install -r requirements.txt` (all versions pinned; see `backend/requirements.txt`)
- **Reproducibility**: `docker-compose up` with provided `Dockerfile`; example APKs in `docs/test_apks/`; expected outputs in `docs/test_apks.md`
- **Re-use limitations**: the Groq API requires an account and a valid API key; the heuristic layer is not available in offline mode in v1.0.0

Acknowledgements

The authors thank the Groq team for API access to Llama 3.3 70B, and the open-source communities behind FastAPI, SQLAlchemy, and ReportLab. This work was carried out as part of the Cybersecurity and Embedded Telecommunications Systems curriculum at the National School of Applied Sciences, Cadi Ayyad University, Marrakech.

References

[1] A. Razaghpanah, R. Nithyanand, N. Vallina-Rodriguez, S. Sundaresan, M. Allman, C. Kreibich, P. Gill, "Apps, Trackers, Privacy, and Regulators: A Global Study of the Mobile Tracking Ecosystem," in *Proc. NDSS Symposium*, San Diego, CA, 2018. Corpus: 2,121 apps, Jan–Mar 2017.

[2] European Parliament and Council, "Regulation (EU) 2016/679 — General Data Protection Regulation," *Official Journal of the European Union*, L 119, pp. 1–88, May 2016. Available: `https://eur-lex.europa.eu/eli/reg/2016/679/oj` (accessed May 2025).

[3] K. Tam, A. Feizollah, N. B. Anuar, R. Salleh, L. Cavallaro, "The Evolution of Android Malware and Android Analysis Techniques," *ACM Computing Surveys*, vol. 49, no. 4, pp. 1–41, 2017.

[4] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, A. N. Sheth, "TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones," *ACM Trans. Computer Systems*, vol. 32, no. 2, pp. 1–29, 2014.

[5] M. Avvenuti, M. Cesarini, A. Vecchio, "Privacy Issues in Android Applications: A Survey," *Future Internet*, vol. 14, no. 7, p. 196, 2022.

[6] A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, D. Wagner, "Android Permissions: User Attention, Comprehension, and Behavior," in *Proc. SOUPS*, Pittsburgh, PA, 2012.

[7] Exodus Privacy, "Exodus Privacy: An Auditing Platform for Android Applications," *exodus-privacy.eu.org*, 2017. Available: `https://exodus-privacy.eu.org` (accessed May 2025).

[8] A. Abraham, "Mobile Security Framework (MobSF)," v3.9, 2015–2024. Available: `https://github.com/MobSF/Mobile-Security-Framework-MobSF` (accessed May 2025).

[9] A. Desnos et al., "Androguard," v3.4.0, 2023. Available: `https://github.com/androguard/androguard` (accessed May 2025).

[10] D. Shrivastava, "InsecureBankv2 — Vulnerable Android Application for Security Professionals," v2.3.1, Apache 2.0, 2013–2024. Available: `https://github.com/dineshshetty/Android-InsecureBankv2` (accessed May 2025).

[11] Groq Inc., "Groq API Documentation — GroqCloud," 2024. Available: `https://console.groq.com/docs` (accessed May 2025).

[12] S. Arzt et al., "FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps," in *Proc. PLDI*, Edinburgh, UK, 2014.

[13] AppCensus Inc., "AppCensus — Privacy Analysis for Mobile Apps," 2019. Available: `https://appcensus.io` (accessed May 2025).

[14] ReportLab Inc., "ReportLab Open-Source PDF Toolkit," v4.1.0, 2024. Available: `https://www.reportlab.com/opensource/` (accessed May 2025).

[15] OWASP Foundation, "OWASP Mobile Top 10 — 2024 Edition," 2024. Available: `https://owasp.org/www-project-mobile-top-10/` (accessed May 2025).

[16] Google, "Permissions on Android — Android Developers," 2024. Available: `https://developer.android.com/guide/topics/permissions/overview` (accessed May 2025).